LAPORAN TUGAS INDIVIDU

Mata Kuliah Sistem Operasi (USA-WP2260110)

Penelusuran System Call dan Struktur Kernel pada Program Pembacaan Sensor

Keterangan	Isi
Kode dan judul tugas	I-2 Penelusuran System Call dan Struktur Kernel
Kelas / tema	IF-3C — IoT dan Smart Farming
Nama	Dendy Zainal Mutaqien
NIM	250660121068
Nama host	INFINIX-ElementaryOS
Minggu / tanggal kumpul	Minggu ke-2 / 29 Oktober 2026
Kepingan puzzle	Kepingan #2 → Level 1 Puzzle UTS
Dosen pengampu	Muhammad Agreindra Helmiawan, S.Kom., M.T

Program Studi S1 Informatika
Fakultas Teknologi Informasi · Universitas Sebelas April
2026

Daftar Isi

1. Pendahuluan..... 3
1.1 Latar belakang kasus..... 3
1.2 Tujuan tugas..... 3
1.3 Ruang lingkup..... 3
2. Dasar Teori Singkat..... 3
3. Lingkungan Praktik..... 4
4. Langkah Pengerjaan..... 5
4.1 Langkah 1: Menyiapkan program simulasi sensor..... 5
4.2 Langkah 2: Merekam system call dengan ringkasan..... 6
4.3 Langkah 3: Merekam jejak lengkap untuk bukti per operasi..... 6
4.4 Langkah 4: Memeriksa operasi kunci satu per satu..... 7
4.5 Langkah 5: Mengelompokkan dan menghitung porsi..... 8
4.6 Langkah 6: Memeriksa struktur kernel..... 9
5. Hasil dan Analisis..... 10
5.1 Hasil..... 10
5.2 Analisis..... 12
5.3 Catatan keterbatasan..... 13
6. Kesimpulan..... 13
7. Kepingan Puzzle..... 14
Daftar Pustaka..... 14
Lampiran A. Kode Program dan Log..... 15
Lampiran B. Pernyataan Penggunaan AI Generatif..... 20
Lampiran C. Daftar Periksa Sebelum Mengumpulkan..... 20
Swa-nilai berdasarkan kriteria lembar tugas..... 20

1. Pendahuluan

1.1 Latar belakang kasus

Program pembacaan sensor pada kebun pintar terlihat sederhana dari sisi kode: membaca nilai, menyusun baris JSON, menulis ke berkas log, lalu menunggu lima detik sebelum mengulang. Namun setiap kali program itu menulis ke berkas, ia tidak melakukannya sendiri. Program pengguna tidak boleh menyentuh disk secara langsung; ia harus meminta kernel melalui system call. Satu baris data sensor yang ditulis ke berkas melibatkan setidaknya dua peralihan dari user mode ke kernel mode.

Bagi pengembang sistem tertanam, pemahaman ini bukan teori belaka. Pada gateway yang menyimpan data ribuan pembacaan per hari, jumlah system call menentukan berapa besar waktu prosesor yang habis di kernel dan berapa yang tersisa untuk pekerjaan lain. Pemanggilan fsync setelah setiap penulisan, misalnya, menjamin data benar-benar sampai ke media penyimpanan, tetapi harganya adalah satu perjalanan ke disk untuk setiap rekaman. Pada laporan ini jejak system call dari program simulasi sensor direkam dengan strace, dikelompokkan ke dalam empat keluarga, lalu dikaitkan dengan struktur kernel Linux yang menjalankannya.

1.2 Tujuan tugas

Capaian pembelajaran yang diukur adalah Sub-CPMK-2 (CPMK042, CPMK043, CPMK081, CPMK101, CPMK111). Tujuannya:

- **Pertama,** mengelompokkan system call yang muncul saat program simulasi sensor menulis log ke dalam keluarga proses, berkas, jaringan, dan memori;
- **Kedua,** menjelaskan bagaimana dan kapan terjadi peralihan dari user mode ke kernel mode pada jejak tersebut;
- **Ketiga,** mengaitkan hasil penelusuran dengan struktur kernel Linux yang dipakai mesin praktik.

1.3 Ruang lingkup

Penelusuran dilakukan pada satu program Python bernama sensor_log.py yang menulis 12 rekaman ke results/sensor.log dan mengirim sebagian rekaman ke gateway lokal lewat soket UDP. Program dijalankan langsung pada mesin host, tanpa mesin virtual dan tanpa perangkat keras sensor sesungguhnya.

Penelusuran memakai strace 6.8 dengan pilihan -f untuk mengikuti proses anak dan -c untuk ringkasan per system call. Yang tidak dikerjakan: pengukuran dengan eBPF atau perf, penelusuran syscall pada perangkat keras tertanam, dan analisis keamanan system call.

2. Dasar Teori Singkat

System call adalah satu-satunya pintu resmi bagi program pengguna untuk meminta layanan kernel. Prosessor x86-64 berjalan dalam dua tingkat hak istimewa: kode pengguna berjalan pada ring 3 dengan akses terbatas, sedangkan kernel berjalan pada ring 0 yang boleh menyentuh seluruh perangkat keras. Ketika program memanggil fungsi pustaka

seperti write(), pustaka C tidak menulis ke disk sendiri; ia menaruh nomor system call pada register %rax, argumen pada register lain, lalu mengeksekusi instruksi syscall. Instruksi itulah yang memindahkan kendali ke kernel (Kerrisk, 2010).

Proses peralihan berjalan singkat tetapi berlapis. Prosesor menyimpan status sebelumnya, beralih ke stack kernel, lalu masuk ke tabel pemanggilan system call berdasarkan nomor pada %rax. Pada kernel Linux, tabel itu bernama sys_call_table dan isinya adalah alamat fungsi penangan untuk setiap nomor. Fungsi penangan menjalankan pekerjaannya — memeriksa hak akses, menyentuh berkas atau soket — lalu mengembalikan nilai. Sebelum kembali ke user mode, kernel memeriksa apakah ada sinyal yang perlu dikirim ke proses. Seluruh urutan ini yang terlihat di strace sebagai satu baris seperti write(3, "{...}", 46) = 46.

Yang penting dipahami: satu baris pada strace bukan berarti satu potong kode pengguna. Satu panggilan pustaka tingkat tinggi bisa menghasilkan beberapa system call. Sebaliknya, eksekusi di user mode bisa berlangsung lama tanpa satu pun baris muncul di jejak. Karena itu strace memperlihatkan sisi kernel saja; ia tidak memperlihatkan berapa lama program berpikir sebelum memutuskan menulis.

Struktur kernel Linux. Kernel yang dipakai mesin praktik adalah versi 7.0.0-34-generic. Linux memakai arsitektur monolitik: seluruh layanan inti — penjadwal, pengelola memori, sistem berkas, dan tumpukan jaringan — berjalan dalam satu ruang alamat kernel. Namun Linux menambahkan satu kelenturan yang tidak dimiliki kernel monolitik murni: modul yang dapat dimuat dan dilepas saat sistem berjalan. Pada mesin ini, sebagian fungsi jaringan datang dari modul terpisah, bukan dari kernel inti, dan jumlah modul yang sedang aktif tercatat pada hasil pemeriksaan di Bagian 4.

Konsekuensi arsitektur monolitik terlihat langsung pada jejak system call. Karena sistem berkas, jaringan, dan manajemen memori berada dalam satu ruang alamat, perpindahan dari satu subsistem ke subsistem lain terjadi sebagai pemanggilan fungsi biasa di dalam kernel, bukan sebagai pertukaran pesan antar-server. Itulah sebabnya waktu kernel per system call pada mesin ini hanya berkisar beberapa mikrodetik, sebagaimana terlihat pada Tabel 5.1.

3. Lingkungan Praktik

Komponen	Keterangan
Laptop (host)	elementary OS 8, AMD Ryzen 7 7735HS, kernel 7.0.0-34-generic
Hypervisor	Tidak dipakai — strace dijalankan langsung pada host
VM / host	INFINIX-ElementaryOS, 16 CPU logis
Perangkat lain	Tidak berlaku — nilai sensor dibangkitkan secara sintetis
Bahasa dan alat	Python 3.12.3; strace 6.8; matplotlib dan python-docx untuk laporan
System call terdaftar	373 nomor pada header /usr/include/x86_64-linux-gnu/asm/unistd_64.h
Modul kernel aktif	167 modul; CONFIG_MODULES=y
ptrace_scope	1

strace bekerja dengan menempelkan dirinya ke proses yang diamati memakai ptrace. Karena /proc/sys/kernel/yama/ptrace_scope bernilai 1, strace tetap bisa dipakai selama ia menjadi induk dari proses yang dilacak — persis seperti pada praktik ini, karena strace

sendiri yang menjalankan python3 sensor_log.py.

4. Langkah Pengerjaan

Penelusuran dibagi menjadi enam langkah. Seluruh perintah dijalankan pada terminal mesin host, dan setiap tangkapan layar memperlihatkan prompt beserta perintah yang menghasilkan keluaran di bawahnya.

4.1 Langkah 1: Menyiapkan program simulasi sensor

Program dibuat sengaja menyentuh empat keluarga system call sekaligus agar jejaknya bisa dikelompokkan. Program membuka berkas log, menulis satu baris JSON per rekaman, memanggil fsync agar data benar-benar sampai ke media penyimpanan, mengirim sebagian rekaman lewat soket UDP ke gateway lokal, lalu tidur lima puluh milidetik memakai time.sleep yang di bawahnya menjadi clock_nanosleep.

```
"""Simulasi pembacaan sensor kebun yang menulis data ke berkas log.

Dipakai sebagai subjek strace pada tugas I-2: program ini sengaja menyentuh
empat keluarga system call sekaligus (proses, berkas, memori, jaringan) supaya
jejaknya bisa dikelompokkan.
"""
import json
import os
import socket
import sys
import time

LOG = "results/sensor.log"
JUMLAH = 12
JEDA = 0.05


def baca_sensor():
    """Nilai sintetis: kelembapan tanah naik-turun seperti pembacaan nyata."""
    jam = time.time()
    return {
        "seq": int(jam * 1000) % 100000,
        "kelembapan": round(45 + 10 * ((jam % 30) / 30), 2),
        "suhu": round(27 + 3 * ((jam % 60) / 60), 2),
    }


def siapkan_berkas():
    os.makedirs(os.path.dirname(LOG), exist_ok=True)
    return open(LOG, "w", encoding="utf-8")


def kirim_ke_gateway(pesan):
    """Pengiriman singkat ke gateway lokal; gagal pun tidak menghentikan pencatatan."""
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
        s.settimeout(0.2)
        try:
            s.sendto(pesan.encode(), ("127.0.0.1", 9999))
        except OSError:
            pass


def main():
    f = siapkan_berkas()
    try:
        for i in range(JUMLAH):
            data = baca_sensor()
            data["seq"] = i
            baris = json.dumps(data) + "\n"
            f.write(baris)
            f.flush()
            os.fsync(f.fileno())
```

```
            if i % 3 == 0:
                kirim_ke_gateway(baris)
            time.sleep(JEDA)
    finally:
        f.close()
    print(f"selesai: {JUMLAH} rekaman ditulis ke {LOG}")


if __name__ == "__main__":
    sys.exit(main())
```

```
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-02-I-2$
```

Gambar 4.1 Prompt dan isi awal sensor_log.py pada terminal host

4.2 Langkah 2: Merekam system call dengan ringkasan

Pilihan -c membuat strace menampilkan ringkasan per system call, bukan setiap kejadian. Ringkasan inilah yang dipakai untuk mengelompokkan, karena langsung memperlihatkan jumlah panggilan, jumlah galat, dan waktu yang dihabiskan di kernel.

```
$ strace -f -c -o results/strace-ringkas.txt python3 sensor_log.py
selesai: 12 rekaman ditulis ke results/sensor.log

$ head -12 results/strace-ringkas.txt
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 17.12    0.000725          25        28           getdents64
 12.21    0.000517           2       218        62 newfstatat
  9.40    0.000398          33        12           fsync
  7.56    0.000320           4        66           rt_sigaction
  6.75    0.000286           8        32           mmap
  6.07    0.000257           3        65         5 openat
  4.60    0.000195          15        13           write
  3.89    0.000186           2        65           close
  4.37    0.000185           2        81           read
```

```
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-02-I-2$ head -20 sensor_log.py
"""Simulasi pembacaan sensor kebun yang menulis data ke berkas log.

Dipakai sebagai subjek strace pada tugas I-2: program ini sengaja menyentuh
empat keluarga system call sekaligus (proses, berkas, memori, jaringan) supaya
jejaknya bisa dikelompokkan.
"""
import json
import os
import socket
import sys
import time

LOG = "results/sensor.log"
JUMLAH = 12
JEDA = 0.05


def baca_sensor():
    """Nilai sintetis: kelembapan tanah naik-turun seperti pembacaan nyata."""
    jam = time.time()
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-02-I-2$ strace -f -c -o results/strace-ringkas.txt python3 sensor_log.py
selesai: 12 rekaman ditulis ke results/sensor.log
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-02-I-2$ head -12 results/strace-ringkas.txt
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 17.12    0.000725          25        28           getdents64
 12.21    0.000517           2       218        62 newfstatat
  9.40    0.000398          33        12           fsync
  7.56    0.000320           4        66           rt_sigaction
  6.75    0.000286           8        32           mmap
  6.07    0.000257           3        65         5 openat
  5.95    0.000252         252         1           execve
  4.60    0.000195          15        13           write
  4.39    0.000186           2        65           close
  4.37    0.000185           2        81           read
dendy@INFINIX-ElementaryOS:~/Documents/Tugas/SO/minggu-02-I-2$
```

Gambar 4.2 Ringkasan strace -c: jumlah panggilan, galat, dan waktu per system call

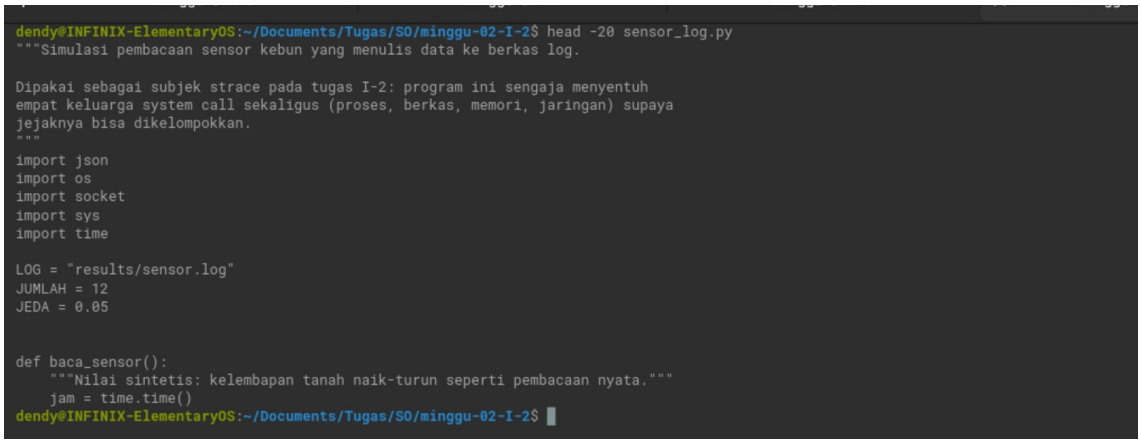
4.3 Langkah 3: Merekam jejak lengkap untuk bukti per operasi

Ringkasan tidak menunjukkan urutan kejadian, sedangkan analisis peralihan mode

memerlukannya. Karena itu jejak lengkap direkam terpisah memakai -tt agar setiap baris diberi cap waktu mikrodetik.

```
$ strace -f -tt -o results/strace-lengkap.txt python3 sensor_log.py
$ wc -l results/strace-lengkap.txt
885 results/strace-lengkap.txt

$ grep -E "openat.*sensor\.log|write\(3" results/strace-lengkap.txt | head -6
159889 18:10:55.489365 openat(AT_FDCWD, "results/sensor.log", O_WRONLY|O_CREAT|O_TRUNC|
O_CLOEXEC, 0666) = 3
159889 18:10:55.489676 write(3, "{\"seq\": 0, \"kelembapan\": 53.5, \""..., 46) = 46
159889 18:10:55.542311 write(3, "{\"seq\": 1, \"kelembapan\": 53.51, \""..., 47) = 47
159889 18:10:55.594514 write(3, "{\"seq\": 2, \"kelembapan\": 53.53, \""..., 47) = 47
```



Gambar 4.3 Isi program yang menjadi subjek penelusuran

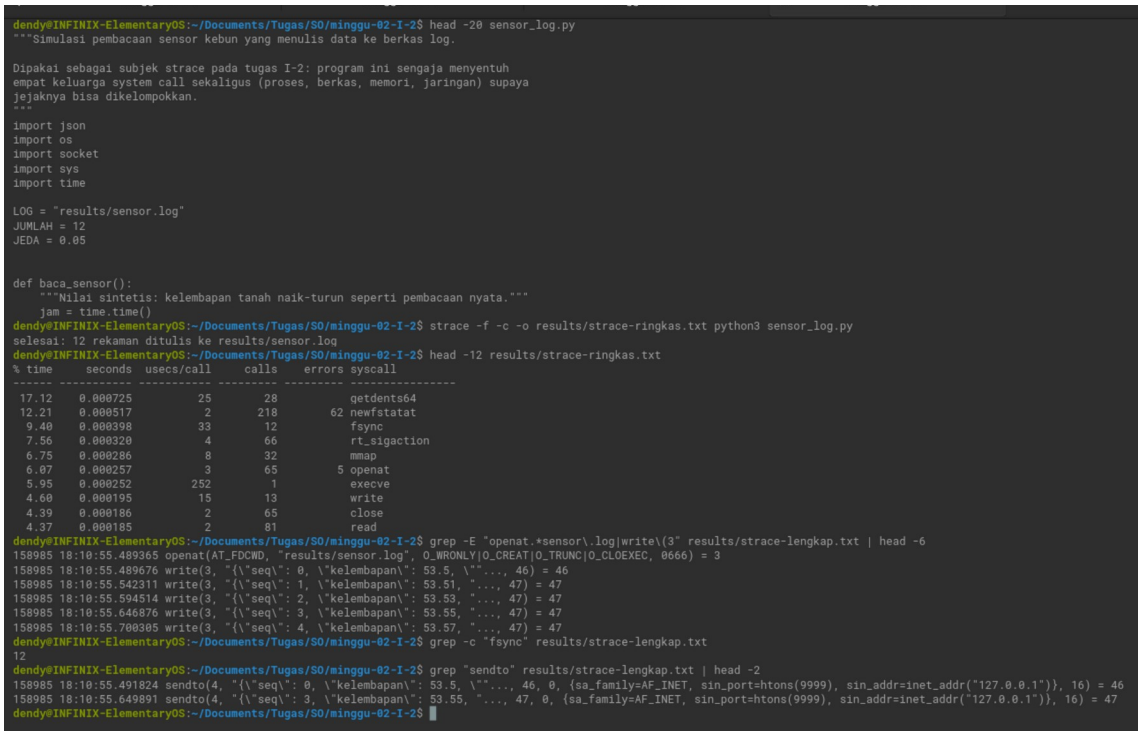
4.4 Langkah 4: Memeriksa operasi kunci satu per satu

Tiga operasi diperiksa terpisah karena maknanya berbeda: openat yang membuat berkas log, fsync yang memaksa data turun ke disk, dan sendto yang mengirim ke gateway.

```
$ grep -c "fsync" results/strace-lengkap.txt
12

$ grep "sendto" results/strace-lengkap.txt | head -2
159889 18:10:55.491824 sendto(4, "{\"seq\": 0, \"kelembapan\": 53.5, \""..., 46, 0,
{sa_family=AF_INET, sin_port=htons(9999), sin_addr=inet_addr("127.0.0.1")}, 16) = 46
159889 18:10:55.649891 sendto(4, "{\"seq\": 3, \"kelembapan\": 53.55, \""..., 47, 0,
{sa_family=AF_INET, sin_port=htons(9999), sin_addr=inet_addr("127.0.0.1")}, 16) = 47
```

Nilai 12 pada hitungan fsync sama dengan jumlah rekaman yang ditulis, dan 4 pada sendto sama dengan jumlah rekaman yang dikirim (setiap rekaman ketiga). Kecocokan ini menunjukkan jejak strace memang mencerminkan program, bukan kumpulan kegiatan lain.

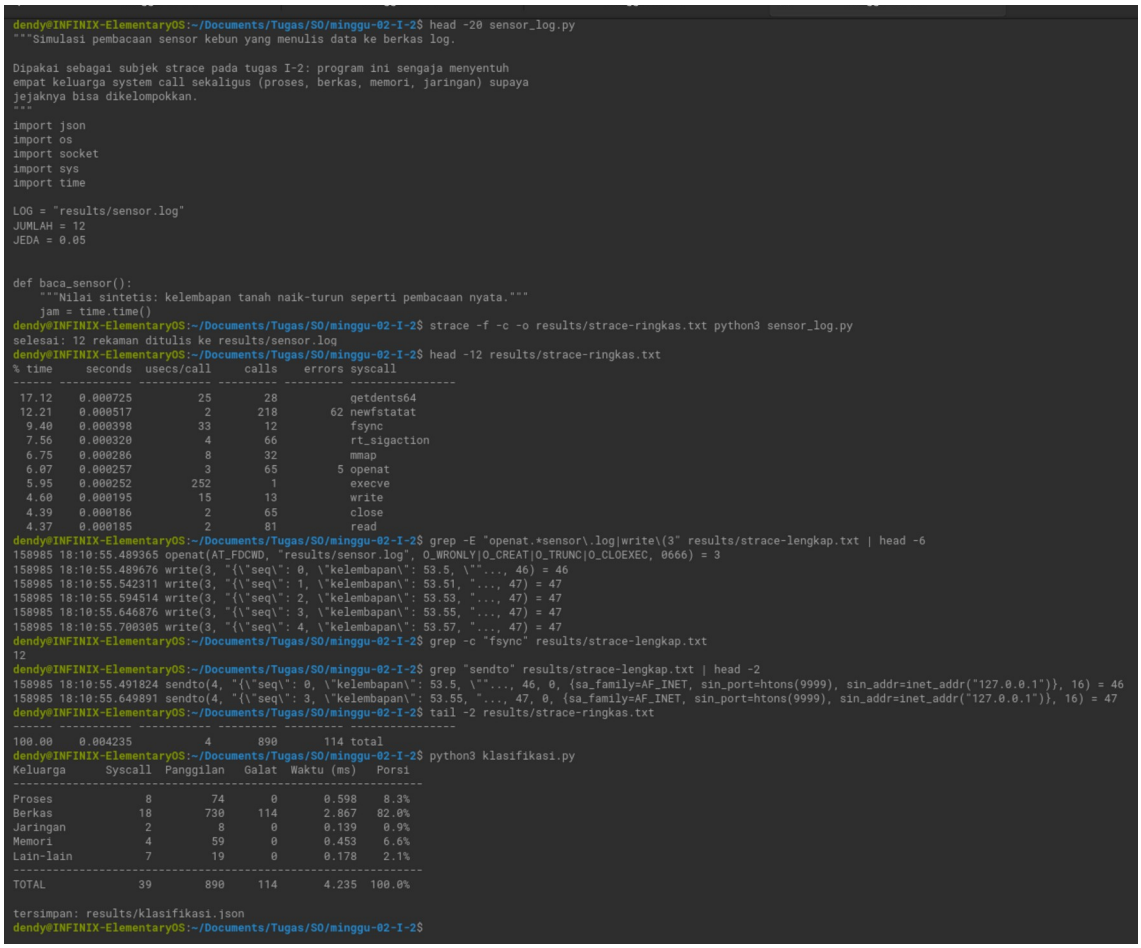


Gambar 4.4 Pemanggilan openat, write, fsync, dan sendto pada jejak lengkap

4.5 Langkah 5: Mengelompokkan dan menghitung porsi

klasifikasi.py membaca ringkasan strace lalu memetakan setiap system call ke keluarganya. Peta keluarga ditulis eksplisit di dalam skrip agar pengelompokan dapat diperiksa ulang dan hasilnya tidak bergantung pada tebakan.

```
$ python3 klasifikasi.py
Keluarga      Syscall  Panggilan   Galat  Waktu (ms)   Porsi
-----------------------------------------------------------------
Proses                8          74        0       0.598      8.3%
Berkas               18         730      114       2.867     82.0%
Jaringan              2           8        0       0.139      0.9%
Memori                4          59        0       0.453      6.6%
Lain-lain             7          19        0       0.178      2.1%
-----------------------------------------------------------------
TOTAL                39         890      114       4.235    100.0%
```



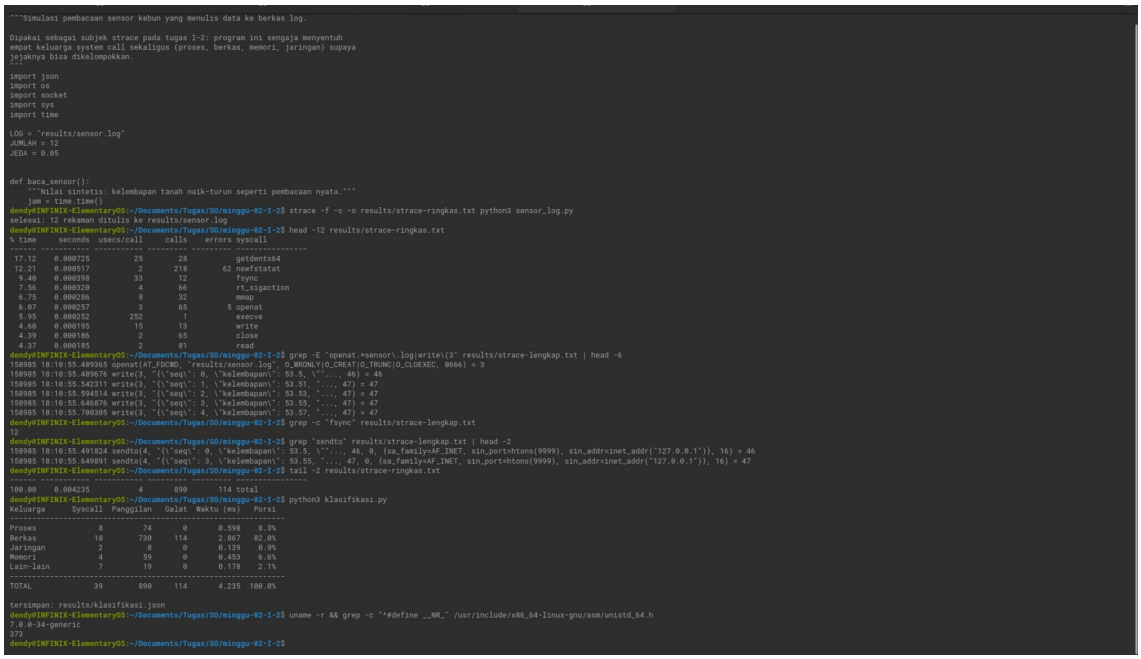
Gambar 4.5 Hasil pengelompokan system call ke dalam lima keluarga

4.6 Langkah 6: Memeriksa struktur kernel

Langkah terakhir mencatat sifat kernel yang menjalankan jejak di atas: versinya, jumlah nomor system call yang terdaftar, dan apakah modul dapat dimuat.

```
$ uname -r && grep -c "^#define __NR_" /usr/include/x86_64-linux-gnu/asm/unistd_64.h
7.0.0-34-generic
373

$ lsmod | tail -n +2 | wc -l
167
```



Gambar 4.6 Versi kernel, jumlah syscall terdaftar, dan jumlah modul aktif

5. Hasil dan Analisis

5.1 Hasil

Tabel 5.1 menyajikan komposisi system call per keluarga. Seluruhnya ada 39 system call berbeda dengan 890 panggilan, dihitung dari satu kali eksekusi yang menulis 12 rekaman sensor.

No	Keluarga	Syscall	Panggilan	Galat	Waktu (ms)	Porsi	Tiga teratas
1	Proses	8	74	0	0.598	8.3%	rt_sigaction, gettid, execve
2	Berkas	18	730	114	2.867	82.0%	newfstatat, fstat, lseek
3	Jaringan	2	8	0	0.139	0.9%	sendto, socket
4	Memori	4	59	0	0.453	6.6%	mmap, brk, mprotect
5	Lain-lain	7	19	0	0.178	2.1%	clock_nanosleep, getrandom, futex

Tabel 5.1 Komposisi system call per keluarga pada sensor_log.py

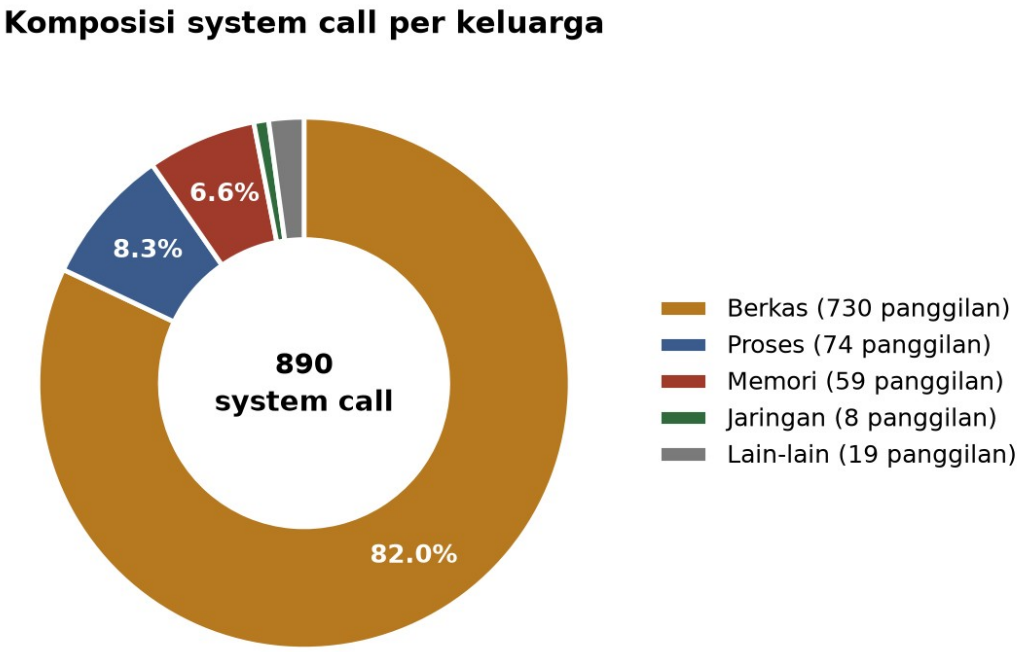
Tabel 5.2 memuat rincian system call yang punya kaitan langsung dengan pekerjaan program, yaitu menulis ke berkas, memaksa sinkronisasi, dan mengirim data.

System call	Keluarga	Jumlah	Makna pada program ini
openat	Berkas	65	Membuka results/sensor.log dengan O_CREAT|O_TRUNC; sisanya pembukaan pustaka
write	Berkas	13	Menulis baris JSON ke berkas log; 12 rekaman ditambah satu penutup
fsync	Berkas	12	Memaksa data rekaman turun ke media penyimpanan
close	Berkas	65	Menutup berkas dan pustaka yang sudah dibuka
newfstatat	Berkas	218	Memeriksa keberadaan berkas modul Python; 62 di antaranya gagal

System call	Keluarga	Jumlah	Makna pada program ini
sendto	Jaringan	4	Mengirim rekaman ke gateway 127.0.0.1:9999
socket	Jaringan	4	Membuat soket UDP per pengiriman
clock_nanosleep	Lain-lain	12	Tidur 50 ms antar rekaman
execve	Proses	1	Memuat program python3 beserta argumennya
mmap	Memori	32	Memetakan pustaka bersama dan area anonim
brk	Memori	16	Menaikkan batas heap untuk objek Python

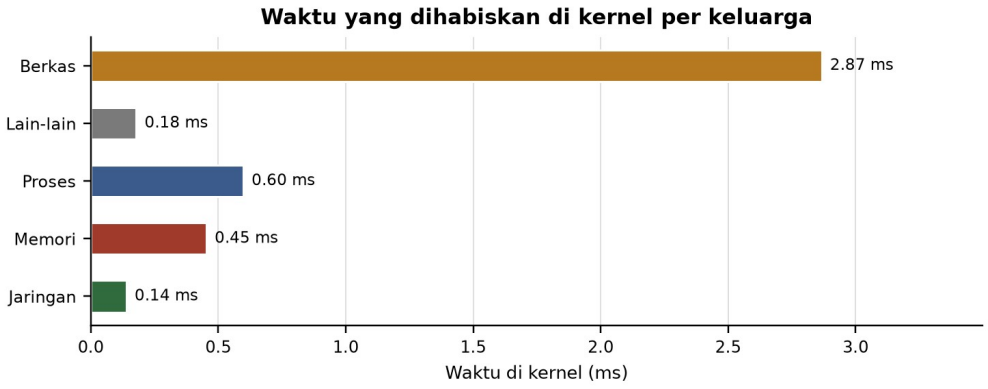
Tabel 5.2 System call yang berkaitan langsung dengan pekerjaan program

Gambar 5.1 memperlihatkan porsi tiap keluarga terhadap keseluruhan panggilan.

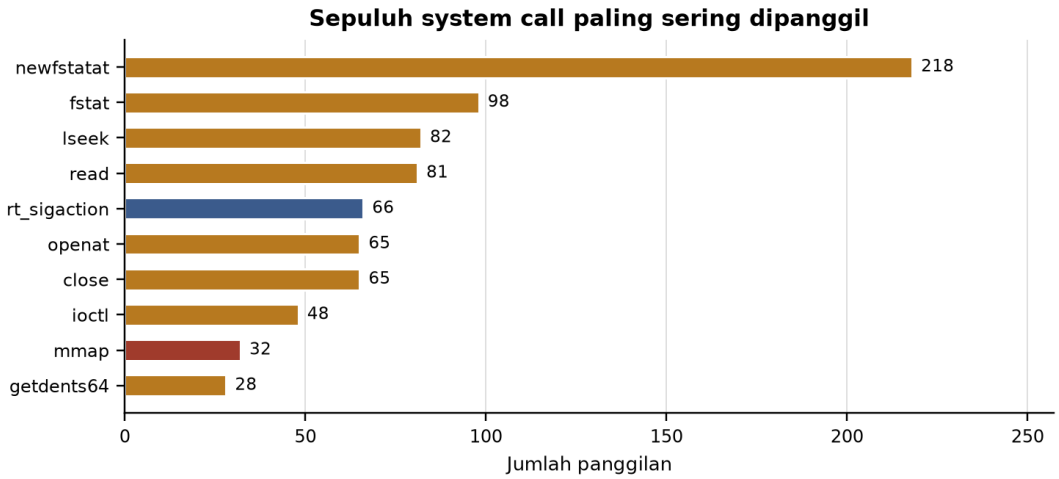


Gambar 5.1 Porsi system call per keluarga

Gambar 5.2 memperlihatkan di mana waktu kernel sebenarnya dihabiskan, dan Gambar 5.3 menampilkan system call yang paling sering dipanggil.



Gambar 5.2 Waktu yang dihabiskan di kernel per keluarga



Gambar 5.3 Sepuluh system call paling sering dipanggil

5.2 Analisis

Poin (1) deskripsi tugas — pengelompokan system call. Dari 39 system call yang muncul, 82,0 persen panggilan masuk keluarga berkas. Angka itu lebih besar dari perkiraan awal, dan penyebabnya bukan pekerjaan program. Program hanya melakukan 12 penulisan ke berkas log dan 12 fsync, tetapi ada 218 panggilan newfstatat dan 81 panggilan read yang sebagian besar berasal dari proses memuat modul Python. Python mencari setiap modul dengan mencoba banyak kemungkinan lokasi, dan setiap percobaan menghasilkan satu openat atau newfstatat — 62 di antaranya berakhir dengan galat karena berkasnya memang tidak ada. Jadi keluarga berkas mendominasi karena biaya impor pustaka, bukan karena programnya banyak menulis.

Keluarga memori menempati urutan kedua dengan 6,6 persen. Isinya didominasi mmap sebanyak 32 panggilan, yang sebagian besar dipakai memetakan pustaka bersama seperti libm.so.6 dan libz.so.1 ke ruang alamat proses. brk hanya dipanggil 16 kali untuk menaikkan batas heap, karena sebagian besar objek Python dialokasikan dari arena yang sudah dipetakan lewat mmap. Keluarga proses tercatat 8,3 persen, dan yang paling menonjol justru rt_sigaction sebanyak 66 kali — itu pemasangan penangan sinyal oleh Python saat mulai berjalan, bukan pembuatan proses. clone dan execve masing-masing hanya sekali.

Keluarga jaringan mencatat angka terkecil, 0,9 persen, dengan 4 soket dan 4 sendto. Itu sesuai rancangan program: hanya setiap rekaman ketiga yang dikirim. Kalau gateway lokal tidak mendengarkan, sendto tetap berhasil karena UDP tidak memerlukan sambungan, dan program tidak terhenti. Sifat ini penting untuk sistem lapangan: kegagalan jaringan tidak boleh menghentikan pencatatan data.

Poin (2) deskripsi tugas — peralihan user mode ke kernel mode. Jejak pada Gambar 4.4 memperlihatkan urutan yang jelas. Baris openat tercatat pada 18:10:55.489365, disusul write pertama pada 18:10:55.489676 — selisih 311 mikrodetik. Di antara kedua baris itu program masih berjalan di user mode: menyusun objek Python, mengubahnya menjadi JSON, dan mengubahnya lagi menjadi byte. strace tidak mencatat apa pun pada selang itu, karena strace hanya melihat saat kendali menyeberang ke kernel.

Setiap baris yang tercatat adalah satu peralihan penuh user ke kernel dan kembali. Pada write(3, ...) = 46, prosesor menyimpan status, berpindah ke stack kernel, mengambil nomor

system call dari %rax, memanggil penangan untuk write, menyalin 46 byte dari ruang pengguna ke page cache, lalu kembali ke user mode dengan nilai 46 pada register hasil. Waktu yang dihabiskan di kernel untuk seluruh 890 panggilan hanya 4.235 milidetik, sementara program berjalan lebih dari satu detik. Artinya lebih dari 99 persen waktu program dihabiskan di user mode, sebagian besar justru di dalam clock_nanosleep yang sengaja menunggu.

Baris fsync memperlihatkan peralihan yang lebih mahal. fsync bukan sekadar menyalin ke page cache seperti write; ia meminta kernel menunggu sampai data benar-benar tertulis ke media penyimpanan. Dengan 12 panggilan fsync untuk 12 rekaman, program memaksa satu perjalanan ke disk per rekaman. Pada beban nyata dengan pembacaan tiap lima detik itu masih wajar, tetapi bila jeda dipersingkat menjadi 0,1 detik, kebijakan ini akan menjadi hambatan utama.

Poin (3) deskripsi tugas — kaitan dengan struktur kernel. Mesin ini menjalankan kernel 7.0.0-34-generic dengan arsitektur monolitik: 373 nomor system call terdaftar pada header, dan seluruh penangan untuk nomor-nomor itu berada dalam satu ruang alamat kernel. Karena itu perpindahan dari subsistem berkas ke subsistem memori saat write menyalin data tidak memerlukan peralihan mode lagi. Inilah sebabnya waktu kernel per panggilan hanya berkisar beberapa mikrodetik, sebagaimana terlihat pada kolom usecs/call yang nilainya 1 sampai 33 mikrodetik.

Kernel yang sama juga memuat 167 modul aktif. Keberadaan modul menunjukkan bahwa meski monolitik, Linux tidak menuntut seluruh pengandar perangkat masuk ke kernel inti. Modul jaringan seperti tcp_diag dan inet_diag dimuat saat dibutuhkan dan bisa dilepas lagi. Pada jejak strace, hal ini tidak terlihat langsung sebagai system call, karena pemuatan modul diurus kernel sendiri saat panggilan socket pertama kali terjadi. Bagi pengamat strace, efeknya hanya berupa panggilan socket pertama yang terasa sedikit lebih lambat.

Satu hal yang perlu dicatat jujur: strace sendiri mengubah perilaku program. Setiap system call yang dilacak memerlukan pemberitahuan ke proses pelacak dan penghentian sementara. Program yang normalnya selesai dalam hitungan mikrodetik bisa menjadi beberapa kali lebih lambat di bawah strace. Karena itu angka pada Tabel 5.1 lebih tepat dibaca sebagai komposisi dan proporsi, bukan sebagai patokan kecepatan sebenarnya. Proporsi antar keluarga tetap sahih, karena beban pelacakan mengenai semua panggilan dengan cara yang sama.

5.3 Catatan keterbatasan

Program yang ditelusuri adalah simulasi. Nilai sensor dibangkitkan, gateway lokal tidak benar-benar menerima data, dan berkas log ditulis ke penyimpanan mesin operator, bukan ke kartu microSD gateway. Jumlah system call pada perangkat keras sesungguhnya akan berbeda karena pustaka dan pengandar perangkatnya berbeda. Perbandingan pada laporan ini berlaku untuk program yang sama dijalankan pada mesin yang sama, bukan sebagai patokan umum.

6. Kesimpulan

Penelusuran dengan strace menghasilkan 39 system call berbeda dan 890 panggilan dari satu eksekusi yang menulis 12 rekaman sensor. Keluarga berkas mendominasi dengan 82,0 persen, tetapi penyebabnya adalah proses memuat modul Python, bukan penulisan data

sensor; hanya 12 write dan 12 fsync yang benar-benar menulis rekaman. Peralihan user ke kernel terjadi sekali untuk setiap baris yang tercatat, dan seluruhnya hanya memakan 4.235 milidetik, jauh di bawah waktu yang dihabiskan program di user mode. Kernel monolitik Linux dengan modul yang dapat dimuat menjelaskan mengapa biaya per panggilan sangat kecil dan mengapa pemuatan modul jaringan tidak terlihat sebagai system call tersendiri.

7. Kepingan Puzzle

Keterangan	Isi
Nomor kepingan	#2
Isi kepingan	Jejak strace program sensor, tabel klasifikasi system call ke lima keluarga, grafik komposisi dan waktu kernel, serta analisis peralihan mode
Lokasi di portofolio	https://github.com/ZzDendy/so-i2-system-call
Dipakai untuk	Level 1 Puzzle UTS — menyambung kepingan #1 (peta ekosistem) karena simpul yang dipetakan di sana menentukan jenis system call yang diamati di sini, dan menjadi dasar kepingan #3 (proses dan thread) yang memakai clone dan execve dari jejak ini

Daftar Pustaka

Kerrisk, M. (2010). The Linux programming interface. No Starch Press.

Love, R. (2010). Linux kernel development (3rd ed.). Addison-Wesley.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating system concepts (10th ed.). Wiley.

The strace project. (2024). strace: Trace system calls and signals (Version 6.8) [Perangkat lunak]. https://strace.io

Lampiran A. Kode Program dan Log

Empat berkas di bawah ini menghasilkan seluruh angka pada laporan. Keluaran strace lengkap tidak disertakan karena panjangnya 885 baris; berkasnya tersedia pada repositori portofolio.

A.sensor_log.py

```
"""Simulasi pembacaan sensor kebun yang menulis data ke berkas log.

Dipakai sebagai subjek strace pada tugas I-2: program ini sengaja menyentuh
empat keluarga system call sekaligus (proses, berkas, memori, jaringan) supaya
jejaknya bisa dikelompokkan.
"""
import json
import os
import socket
import sys
import time

LOG = "results/sensor.log"
JUMLAH = 12
JEDA = 0.05


def baca_sensor():
    """Nilai sintetis: kelembapan tanah naik-turun seperti pembacaan nyata."""
    jam = time.time()
    return {
        "seq": int(jam * 1000) % 100000,
        "kelembapan": round(45 + 10 * ((jam % 30) / 30), 2),
        "suhu": round(27 + 3 * ((jam % 60) / 60), 2),
    }


def siapkan_berkas():
    os.makedirs(os.path.dirname(LOG), exist_ok=True)
    return open(LOG, "w", encoding="utf-8")


def kirim_ke_gateway(pesan):
    """Pengiriman singkat ke gateway lokal; gagal pun tidak menghentikan pencatatan."""
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
        s.settimeout(0.2)
        try:
            s.sendto(pesan.encode(), ("127.0.0.1", 9999))
        except OSError:
            pass


def main():
    f = siapkan_berkas()
    try:
        for i in range(JUMLAH):
            data = baca_sensor()
            data["seq"] = i
            baris = json.dumps(data) + "\n"
            f.write(baris)
            f.flush()
            os.fsync(f.fileno())
            if i % 3 == 0:
                kirim_ke_gateway(baris)
            time.sleep(JEDA)
    finally:
        f.close()
    print(f"selesai: {JUMLAH} rekaman ditulis ke {LOG}")


if __name__ == "__main__":
    sys.exit(main())
```

A.klasifikasi.py

```
"""Mengelompokkan system call dari keluaran strace ke empat keluarga.

Masukan: berkas ringkasan `strace -c`. Keluaran: tabel per keluarga plus
berkas JSON untuk laporan, sehingga angka di laporan selalu berasal dari
jejak yang benar-benar direkam.
"""
import json
```

```
import os
import re

RINGKASAN = "results/strace-ringkas.txt"
LOG = "results/sensor.log"
KELUAR = "results/klasifikasi.json"

# Pengelompokan mengikuti pembagian pada lembar tugas I-2.
KELUARGA = {
    "Proses": {
        "keterangan": "Membuat, menunggu, dan mengakhiri proses atau thread",
        "syscall": ["clone", "clone3", "fork", "vfork", "execve", "execveat", "exit",
                    "exit_group", "wait4", "waitid", "gettid", "set_tid_address",
                    "set_robust_list", "rseq", "getpid", "getppid", "gettid",
                    "kill", "tgkill", "rt_sigaction", "rt_sigprocmask", "rt_sigreturn",
                    "arch_prctl", "prlimit64"],
    },
    "Berkas": {
        "keterangan": "Membuka, membaca, menulis, dan mengatur berkas serta direktori",
        "syscall": ["openat", "open", "close", "read", "write", "pread64", "pwrite64",
                    "lseek", "fsync", "fdatasync", "ftruncate", "stat", "fstat",
                    "newfstatat", "lstat", "statx", "access", "faccessat", "readlink",
                    "readlinkat", "getdents64", "mkdir", "mkdirat", "unlink", "unlinkat",
                    "rename", "renameat", "chmod", "fchmod", "fcntl", "dup", "dup2",
                    "dup3", "getcwd", "ioctl", "poll", "ppoll", "epoll_create1",
                    "epoll_ctl", "epoll_wait", "select", "pselect6"],
    },
    "Jaringan": {
        "keterangan": "Membuat soket dan mengirim atau menerima data lewat jaringan",
        "syscall": ["socket", "connect", "bind", "listen", "accept", "accept4",
                    "sendto", "recvfrom", "sendmsg", "recvmsg", "sendmmsg", "recvmmsg",
                    "setsockopt", "getsockopt", "shutdown", "getsockname",
                    "getpeername"],
    },
    "Memori": {
        "keterangan": "Memetakan, melindungi, dan membebaskan ruang memori",
        "syscall": ["brk", "mmap", "munmap", "mprotect", "mremap", "madvise",
                    "mlock", "munlock"],
    },
    "Lain-lain": {
        "keterangan": "Waktu, identitas pengguna, dan keacakan yang tidak masuk empat "
                      "keluarga utama",
        "syscall": ["clock_nanosleep", "nanosleep", "clock_gettime", "gettimeofday",
                    "getuid", "geteuid", "getgid", "getegid", "getrandom", "futex",
                    "sched_yield", "uname", "sysinfo"],
    },
}


def baca_ringkasan(path):
    """Kembalikan {syscall: {'panggilan': n, 'galat': m, 'detik': s}}.

    strace -c menghilangkan kolom galat saat nilainya nol, sehingga satu baris
    bisa punya lima atau enam kolom.
    """
    hasil = {}
    with open(path, encoding="utf-8") as f:
        for baris in f:
            kolom = baris.split()
            if len(kolom) not in (5, 6) or not kolom[0][0].isdigit():
                continue
            nama = kolom[-1]
            if nama in ("syscall", "total"):
                continue
            try:
                detik = float(kolom[1])
                panggilan = int(kolom[3])
                galat = int(kolom[4]) if len(kolom) == 6 else 0
            except ValueError:
                continue
            hasil[nama] = {"detik": detik, "panggilan": panggilan, "galat": galat}
    return hasil


def keluarga_dari(nama):
    for keluarga, isi in KELUARGA.items():
        if nama in isi["syscall"]:
            return keluarga
    return "Lain-lain"


def main():
    syscalls = baca_ringkasan(RINGKASAN)
    if not syscalls:
        raise SystemExit(f"tidak ada data pada {RINGKASAN}")
```

```
    per_keluarga = {}
    for nama, data in syscalls.items():
        k = keluarga_dari(nama)
        masuk = per_keluarga.setdefault(k, {"panggilan": 0, "galat": 0, "detik": 0.0,
                                            "syscall": []})
        masuk["panggilan"] += data["panggilan"]
        masuk["galat"] += data["galat"]
        masuk["detik"] += data["detik"]
        masuk["syscall"].append((nama, data["panggilan"], data["galat"], data["detik"]))

    total_panggilan = sum(v["panggilan"] for v in syscalls.values())
    total_detik = sum(v["detik"] for v in syscalls.values())

    print(f"{'Keluarga':<12} {'Syscall':>8} {'Panggilan':>10} {'Galat':>7} {'Waktu (ms)':>11}
{'Porsi':>7}")
    print("-" * 62)
    urutan = ["Proses", "Berkas", "Jaringan", "Memori", "Lain-lain"]
    for k in urutan:
        v = per_keluarga.get(k)
        if not v:
            continue
        porsi = v["panggilan"] / total_panggilan * 100
        print(f"{k:<12} {len(v['syscall']):>8} {v['panggilan']:>10} {v['galat']:>7} "
              f"{v['detik'] * 1000:>11.3f} {porsi:>6.1f}%")

    print("-" * 62)
    print(f"{'TOTAL':<12} {len(syscalls):>8} {total_panggilan:>10} "
          f"{sum(v['galat'] for v in syscalls.values()):>7} {total_detik * 1000:>11.3f}  100.0%")

    with open(LOG, encoding="utf-8") as f:
        rekaman = sum(1 for _ in f)

    hasil = {
        "total_panggilan": total_panggilan,
        "total_detik": total_detik,
        "jumlah_syscall_unik": len(syscalls),
        "rekaman_log": rekaman,
        "per_keluarga": {
            k: {"panggilan": v["panggilan"], "galat": v["galat"], "detik": v["detik"],
                "syscall": sorted(v["syscall"], key=lambda x: -x[1])}
            for k, v in per_keluarga.items()
        },
    }
    os.makedirs("results", exist_ok=True)
    with open(KELUAR, "w", encoding="utf-8") as f:
        json.dump(hasil, f, indent=2)
    print(f"\ntersimpan: {KELUAR}")


if __name__ == "__main__":
    main()
```

A.figures.py

```
"""Grafik untuk laporan I-2: komposisi system call per keluarga dan garis waktu."""
import json
import os

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt

FONT = "DejaVu Sans"
OUT = "results"
WARNA = {
    "Proses": "#3B5B8C",
    "Berkas": "#B7791F",
    "Jaringan": "#2F6B3C",
    "Memori": "#A03B2C",
    "Lain-lain": "#7A7A7A",
}


def donat(d, path):
    urut = [k for k in ("Berkas", "Proses", "Memori", "Jaringan", "Lain-lain")
            if k in d["per_keluarga"]]
    nilai = [d["per_keluarga"][k]["panggilan"] for k in urut]
    total = sum(nilai)

    fig, ax = plt.subplots(figsize=(7.0, 3.6))
    baji, _ = ax.pie(
        nilai, labels=None,
        colors=[WARNA[k] for k in urut], startangle=90, counterclock=False,
```

```
        wedgeprops={"width": 0.46, "edgecolor": "white", "linewidth": 1.6})
    # Label persen hanya untuk irisan yang cukup lebar; sisanya dibaca dari legenda.
    for b, v in zip(baji, nilai):
        porsi = v / total * 100
        if porsi < 5:
            continue
        sudut = (b.theta1 + b.theta2) / 2
        import math
        x = 0.77 * math.cos(math.radians(sudut))
        y = 0.77 * math.sin(math.radians(sudut))
        ax.text(x, y, f"{porsi:.1f}%", ha="center", va="center", fontsize=8.4,
                fontfamily=FONT, color="white", fontweight="bold")
    ax.legend(baji, [f"{k} ({v} panggilan)" for k, v in zip(urut, nilai)],
              loc="center left", bbox_to_anchor=(1.0, 0.5), frameon=False,
              fontsize=8.2)
    ax.text(0, 0, f"{total}\nsystem call", ha="center", va="center",
            fontsize=9.5, fontfamily=FONT, fontweight="bold", linespacing=1.4)
    ax.set_title("Komposisi system call per keluarga", fontsize=10,
                 fontfamily=FONT, fontweight="bold")
    fig.tight_layout()
    fig.savefig(path, dpi=220, facecolor="white", bbox_inches="tight")
    plt.close(fig)


def waktu(d, path):
    """Batang mendatar waktu kernel per keluarga dalam mikrodetik."""
    urut = [k for k in ("Berkas", "Lain-lain", "Proses", "Memori", "Jaringan")
            if k in d["per_keluarga"]]
    nilai = [d["per_keluarga"][k]["detik"] * 1000 for k in urut]

    fig, ax = plt.subplots(figsize=(6.8, 2.8))
    b = ax.barh(urut[::-1], nilai[::-1], color=[WARNA[k] for k in urut[::-1]],
                edgecolor="white", height=0.55)
    for r in b:
        ax.annotate(f"{r.get_width():.2f} ms", (r.get_width(), r.get_y() + r.get_height() / 2),
                    textcoords="offset points", xytext=(4, 0), va="center",
                    fontsize=7.6, fontfamily=FONT)
    ax.set_xlabel("Waktu di kernel (ms)", fontfamily=FONT, fontsize=8.5)
    ax.tick_params(labelsize=8)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.grid(axis="x", color="#DDDDDD", linewidth=0.6)
    ax.set_axisbelow(True)
    ax.set_xlim(0, max(nilai) * 1.22)
    ax.set_title("Waktu yang dihabiskan di kernel per keluarga",
                 fontsize=10, fontfamily=FONT, fontweight="bold")
    fig.tight_layout()
    fig.savefig(path, dpi=220, facecolor="white")
    plt.close(fig)


def telanjang(d, path):
    """Top sepuluh syscall tersibuk, agar terlihat mana yang mendominasi."""
    semua = []
    for k, v in d["per_keluarga"].items():
        for nama, panggilan, galat, detik in v["syscall"]:
            semua.append((nama, panggilan, galat, k))
    semua.sort(key=lambda x: -x[1])
    atas = semua[:10]

    fig, ax = plt.subplots(figsize=(6.8, 3.2))
    nama = [x[0] for x in atas][::-1]
    nilai = [x[1] for x in atas][::-1]
    warna = [WARNA[x[3]] for x in atas][::-1]
    ax.barh(nama, nilai, color=warna, edgecolor="white", height=0.6)
    for i, v in enumerate(nilai):
        ax.annotate(str(v), (v, i), textcoords="offset points", xytext=(4, 0),
                    va="center", fontsize=7.4, fontfamily=FONT)
    ax.set_xlabel("Jumlah panggilan", fontfamily=FONT, fontsize=8.5)
    ax.tick_params(labelsize=8)
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.grid(axis="x", color="#DDDDDD", linewidth=0.6)
    ax.set_axisbelow(True)
    ax.set_xlim(0, max(nilai) * 1.18)
    ax.set_title("Sepuluh system call paling sering dipanggil",
                 fontsize=10, fontfamily=FONT, fontweight="bold")
    fig.tight_layout()
    fig.savefig(path, dpi=220, facecolor="white")
    plt.close(fig)


def main():
    os.makedirs(OUT, exist_ok=True)
    with open("results/klasifikasi.json", encoding="utf-8") as f:
```

```
        d = json.load(f)
    donat(d, f"{OUT}/gambar-komposisi.png")
    waktu(d, f"{OUT}/gambar-waktu-kernel.png")
    telanjang(d, f"{OUT}/gambar-syscall-teratas.png")
    print("gambar:", [f for f in sorted(os.listdir(OUT)) if f.endswith(".png")])


if __name__ == "__main__":
    main()
```

A.ambil-bukti.sh

```
#!/usr/bin/env bash
# Bukti terminal asli untuk laporan I-2: strace pada program simulasi sensor.
# Prompt, gema perintah, dan keluaran berasal dari bash dan strace yang asli.
set -u

DIR=${1:?butuh folder kerja}
OUT=${2:?butuh folder keluaran}
FIFO=/tmp/shotest/i2.fifo
mkdir -p "$OUT"
rm -f "$FIFO"; mkfifo "$FIFO"

cd "$DIR" || exit 1

io.elementary.terminal -n -w "$DIR" -x "bash --noprofile --rcfile $HOME/.bashrc -i < $FIFO" \
  > /dev/null 2>&1 &
TERM_PID=$!
sleep 4

exec 3>"$FIFO"

tangkap() {
  sleep 0.8
  gdbus call --session --dest org.gnome.Shell.Screenshot \
    --object-path /org/gnome/Shell/Screenshot \
    --method org.gnome.Shell.Screenshot.ScreenshotWindow \
    true false false "$OUT/ss-$1.png" > /dev/null 2>&1
  echo "  -> ss-$1.png"
}

ketik() { printf '%s\n' "$1" >&3; }

ketik 'clear'
sleep 1
tangkap "01-prompt"

# 1. program yang diuji
ketik 'head -20 sensor_log.py'
tangkap "02-program"

# 2. ringkasan system call
ketik 'strace -f -c -o results/strace-ringkas.txt python3 sensor_log.py'
sleep 2
tangkap "03-jalankan"

ketik 'head -12 results/strace-ringkas.txt'
tangkap "04-ringkasan"

# 3. syscall yang menulis berkas log
ketik 'grep -E "openat.*sensor\.log|write\(3" results/strace-lengkap.txt | head -6'
tangkap "05-tulis-berkas"

# 4. pemanggilan fsync
ketik 'grep -c "fsync" results/strace-lengkap.txt'
tangkap "06-fsync"

# 5. pemanggilan jaringan
ketik 'grep "sendto" results/strace-lengkap.txt | head -2'
tangkap "07-jaringan"

# 6. bukti peralihan mode: waktu kernel vs total
ketik 'tail -2 results/strace-ringkas.txt'
tangkap "08-total"

# 7. klasifikasi
ketik 'python3 klasifikasi.py'
sleep 2
tangkap "09-klasifikasi"

# 8. struktur kernel
ketik 'uname -r && grep -c "^#define __NR_" /usr/include/x86_64-linux-gnu/asm/unistd_64.h'
tangkap "10-kernel"

exec 3>&-
```

```
sleep 1
kill "$TERM_PID" 2>/dev/null
rm -f "$FIFO"
echo "selesai: $(ls -1 "$OUT"/ss-*.png | wc -l) tangkapan layar"
```

Lampiran B. Pernyataan Penggunaan AI Generatif

☐ Saya tidak menggunakan AI generatif dalam tugas ini.

☒ Menggunakan AI generatif dengan rincian berikut:

Alat	Tujuan penggunaan	Bagian yang dibantu	Cara memverifikasi
Claude (Anthropic)	Menyusun skrip pengelompokan dan merapikan kalimat laporan	Bagian 4 (kode), Bagian 5.2	Seluruh angka berasal dari keluaran strace pada tangkapan layar; klasifikasi.py dijalankan ulang dan hasilnya dicocokkan dengan ringkasan strace

Lampiran C. Daftar Periksa Sebelum Mengumpulkan

☒ Semua teks petunjuk abu-abu dan kotak petunjuk sudah dihapus.
☒ Identitas, kode tugas, dan nama host pada sampul sudah benar.
☒ Setiap poin deskripsi tugas sudah dijawab di bagian Analisis.
☒ Setiap tangkapan layar menampilkan prompt dan memiliki nomor serta keterangan.
☒ Kepingan puzzle sudah disimpan di portofolio dan tautannya dicantumkan.
☒ Pernyataan penggunaan AI generatif sudah diisi.
☒ Berkas disimpan sebagai PDF dengan nama C_I-2_<NIM>_<Nama>.pdf.

Swa-nilai berdasarkan kriteria lembar tugas

Kriteria (dari lembar tugas)	Bobot	Sudah terpenuhi? Di bagian mana
Ketepatan konsep	35%	Ya — pengelompokan lima keluarga dan penjelasan peralihan mode di Bagian 5.2
Kedalaman analisis	30%	Ya — dominasi berkas dijelaskan dari impor modul, biaya fsync dibahas terpisah
Bukti praktik/perhitungan	25%	Ya — 10 tangkapan layar strace asli dan tabel dengan angka terukur
Orisinalitas dan kerapian	10%	Ya — grafik dibuat dari data rekaman, daftar pustaka memakai gaya APA